

AEGIS Demo Setup

Goal: clone the repo onto a Mac or Windows laptop and have the *whole* AEGIS platform — API, admin dashboard, upload portal, all the sidecars (defacing, PHI detection, QC, classification, protocol compliance, BIDS conversion, synthetic-MRI generation, analytics, SCT) — running with one command so you can demo it end-to-end to a colleague.

Everything runs in Docker containers. You do **not** need to install PostgreSQL, Python, Go, or any of the imaging libraries on your host machine.

Prerequisites

Tool	Why	How to install
Docker Desktop	Runs every service in containers	Download from docker.com . License is free for personal/educational use.
Git	Clone the repo	Mac: <code>brew install git</code> · Windows: git-scm.com (or via Git for Windows installer)
Disk space	First-time image builds total ~6 GB	Docker Desktop → Settings → Resources should allow at least 6 GB
RAM	8 GB is the comfortable minimum, 16 GB is recommended	Docker Desktop → Settings → Resources

Optional (only if you want to send synthetic DICOM studies for the demo):

Tool	Why
Python 3.11+	Runs the <code>aegis-router-send-test</code> helper that pushes test studies
<code>pip install pynetdicom pydicom httpx</code>	One-time install (~50 MB)

One-time setup (5 minutes)

1. Clone the repo

```
git clone https://github.com/aegis-imaging/aegis.git
cd aegis
```

2. Start Docker Desktop

Open Docker Desktop and wait until the whale icon settles into the menu bar (Mac) / system tray (Windows). The `docker info` command should succeed in your terminal.

3. Choose a demo path

There are three options depending on what you want to show. Pick one:

Path A — "Look at the whole AEGIS platform" (recommended first time)

Brings up the full cloud stack: API, admin dashboard, upload portal, every sidecar, Postgres, Mailpit (email capture), DWV viewer.

```
docker compose up -d
```

Wait ~60–90 s on the first run (builds the images). On subsequent runs it's ~5 s.

What you can show:

- Upload portal at <http://localhost:3000> — drag a DICOM study, watch client-side tag de-id happen, click upload.
- Admin dashboard at <http://localhost:3001> — see the study appear, click through to the DWV viewer, see the audit log update.
- Synthetic MRI generator — generate a fake brain MRI study in the dashboard's Studies tab → it flows through classification → defacing → PHI scan → QC → BIDS without you doing anything else.
- Mailpit at <http://localhost:8025> — see the upload-confirmation email AEGIS sent (no real SMTP needed).
- The full system in Docker Desktop — open it and you'll see ~15 containers side by side. Visceral way to convey "this is a real platform."

Path B — "Look how this fits at a hospital" (lighter footprint)

Brings up the on-prem **AEGIS Router** stack: DICOM SCP receiver, de-id pipeline, local quarantine, optional sidecars. Uses **SQLite** for local quarantine — no Postgres at all.

```
cd router
cp .env.example .env
docker compose up -d          # router only, smallest footprint
# or, with face de-id and PHI scrub sidecars:
docker compose --profile defacing --profile phi up -d
```

What you can show:

- The router is what a hospital would install on a single Linux VM (or in your case, Docker Desktop on Mac/Win).
- PACS auto-routing: simulate a hospital pushing a study by running the `aegis-router-send-test` script (see below).
- Local-only mode: leave `CLOUD_RECEIVER_URL` empty in `.env` and studies stay on your machine — nice for showing the privacy posture.

Path C — "Just the upload portal in a browser" (smallest)

If you only want the browser-based upload portal without the rest of the stack:

```
cd frontend/upload-portal
npm install
npm run dev
```

Open <http://localhost:3000>. Won't be able to actually push studies anywhere without the API, but you can show the **heavy-mode** in-browser de-id flow (tag + pixel scrub + face de-id all in the browser).

Sending a test study to your local stack

Once one of the paths above is running, push a synthetic study at it so the demo isn't a blank screen.

Option 1 — synthetic MRI generator (full stack only, no CLI needed)

In the admin dashboard at <http://localhost:3001>:

1. Click **Studies** in the sidebar.
2. Click **Generate synthetic MRI**.
3. Pick a number of slices (20 is fine), click Generate.

The study flows through the whole pipeline automatically — refresh the Studies tab to watch it progress through classification → defacing → PHI scan → QC → BIDS.

Option 2 — DIMSE push (router stack)

If you're on Path B (router), use the included sender:

```
# In a new terminal
cd /path/to/aegis/router
pip install pynetdicom pydicom httpx
./bin/aegis-router-send-test --slices 30
```

This generates a synthetic 30-slice MR study and pushes it via DICOM C-STORE to the router on localhost:11112. The router de-identifies it locally and either forwards to the cloud (if configured) or holds it in `/var/lib/aegis-router/studies/` (if not).

```
# Confirm what the pipeline did
curl http://localhost:8080/audit | jq
```

Option 3 — web upload portal

If you have a real or synthetic DICOM file on disk:

1. Open the upload portal at <http://localhost:3000>.
2. Drag the DICOM file (or folder of files) onto the dropzone.
3. Watch the in-browser tag de-id preview, then click Upload.

For the **heavy mode** demo (in-browser pixel PHI scrub + face de-id), toggle "On-device deep de-identification" in the preview panel before clicking Upload.

Tearing down

```
docker compose down          # stop everything, keep data
docker compose down -v       # stop + delete the Postgres volume too
```

If you took Path B (router):

```
cd router
docker compose down -v
```

Troubleshooting

Symptom	Likely cause	Fix
docker compose up says "no space left on device"	Docker Desktop's disk allocation is too small	Settings → Resources → Disk image size → bump to 64 GB+
Dashboard shows "API unhealthy" for >1 min	Postgres still warming up	Wait another 30 s; check <code>docker compose logs postgres</code>
Upload portal returns 5xx	API not running or DB not ready	<code>docker compose ps</code> to see container state
Port 3000/3001/8080/11112 in use	Another service grabbed it	Either stop the other thing or edit <code>docker-compose.yml</code> to remap
Mac M-series build is slow	First-time arm64 image builds	Subsequent runs use cached layers; usually a one-time cost

For more, see [docs/runbooks/](#).

What's running where

Once `docker compose up` is happy, you'll see these containers in Docker Desktop:

Container	Port	What it is
<code>aegis-api</code>	8080	Go HTTP API, the brain of the system
<code>aegis-admin-dashboard</code>	3001	React UI for image analysts + admins
<code>aegis-upload-portal</code>	3000	React UI for the public-facing upload flow
<code>postgres</code>	5432	App database (users, studies, audit log)
<code>mailpit</code>	1025/8025	Local SMTP capture so emails don't escape
<code>dvw</code>	3005	DICOM Web Viewer for studies in the dashboard
<code>defacing</code>	(internal)	Face de-id sidecar (<code>mri_deface</code> / <code>DeepDefacer</code>)
<code>phi-detection</code>	(internal)	Burned-in PHI scrub (Tesseract OCR + cloud OCR options)
<code>qc-service</code>	(internal)	Automated QC checks
<code>bids-service</code>	(internal)	NIFTI/BIDS conversion (<code>dcm2niix</code>)
<code>classification-service</code>	(internal)	Modality/body-part classifier
<code>protocol-service</code>	(internal)	MRI protocol compliance checker

synth-service	(internal)	Synthetic MRI generator (useful for demos)
analytics-service	(internal)	FreeSurfer / FSL / ANTs / etc.
sct-service	(internal)	Spinal Cord Toolbox
dimse-receiver	11112	DICOM C-STORE SCP (what a PACS would push to)

After the demo

If your colleague is interested in deploying this for their lab, the relevant next steps depend on what they want:

- **Their own laptop** — same instructions as above. Encourage them to clone the repo and try it themselves.
- **A shared server in their lab** — point them at [router/docs/research-deployment.md](#) .
- **An on-prem deployment at a hospital** — point them at [router/docs/spoke-deployment.md](#) .
- **A real cloud deployment** — Terraform modules for GCP, AWS, and Azure live in `terraform/` . The cloud auto-deploys on push to `develop` via Cloud Build (GCP) and GitHub Actions (AWS, Azure).

Built with [Claude Code](#). Questions to support@aegisimaging.ai.